

GUIA VALIDACIONES

Objetivo

Es responsabilidad del backend implementar las validaciones. Estas validaciones pueden ser desde el tipo de los datos al crear una nueva subasta hasta cómo responder a una cliente que intenta acceder a un identificador de subasta que no existe.

Durante el siguiente guión veremos los diferentes tipos de validaciones que existen que podemos hacer en django rest framework.

Enunciado

Configuración del entorno:

1. Activamos el env de django que hemos configurado en sesiones anteriores (**djangoServer**) de la siguiente manera:
 - Abre un terminal de **Comand Shell. No vale PowerShell de Windows.**
 - Localízate en esa carpeta:
`cd djangoServer/Scripts`
 - Activa el entorno virtual:
`Activate`

Validaciones en el modelo

La validación a nivel de modelo es el proceso de asegurar que los datos almacenados en una base de datos cumplen ciertas reglas de integridad y restricciones definidas en el modelo.

¿Cuándo usarlo?

- ☒ Para restricciones universales (por ejemplo, valores mínimos/máximos).
- ☒ Para validaciones automáticas sin importar desde dónde venga la solicitud (admin, API, consola).
- ☒ Cuando necesitas asegurarte de que los datos en la base de datos sean siempre correctos.
- ☒ Si la validación depende de otros datos no disponibles en el modelo.
- ☒ Si necesitas enviar mensajes de error personalizados al usuario final.

¿Cómo implementar validaciones en modelo?

1. Accede al archivo `auctions/models.py` e incluye las dos líneas marcadas en negrita.

```
from django.core.validators import MinValueValidator

class Auction(models.Model):
    stock = models.IntegerField(validators=[MinValueValidator(1)])
    #resto de campos del modelo
```

2. **MinValueValidator()** es un validador que django nos facilita por defecto. Determina el valor mínimo del campo stock a 1 unidad. De esta forma, no podremos crear subastas que tengan stock a 0. Probémoslo.
3. Como se ha realizado un cambio en el modelo será necesario realizar los siguientes comandos:
 - `python manage.py makemigrations`
 - `python manage.py migrate`
4. Arranca el servidor.
 - `python manage.py runserver`
5. Navega a <http://127.0.0.1:8000/api/auctions/> y crea una subasta con stock a 0. Podéis usar los valores que queráis para el resto de campos:



The screenshot shows a web form for creating an auction. The fields are as follows:

- Closing date:** 24/05/2025 21:49
- Title:** Laptop Gamer ASUS
- Description:** Laptop gamer con RTX 3060 y 16GB RAM.
- Price:** 1500
- Rating:** 3
- Stock:** 0 (highlighted in yellow)
- Brand:** ASUS
- Thumbnail:** https://dlcdnwebimgs.asus.com/gain/3D241166-0518-4745-B481-D901886BFD14
- Category:** laptops

A blue "POST" button is located at the bottom right of the form.

6. El servidor nos devolverá el siguiente error:

```
Auction List Create

POST /api/auctions/

HTTP 400 Bad Request
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "stock": [
    "Ensure this value is greater than or equal to 1."
  ]
}
```

Closing date	24/05/2025 21:49
Title	Laptop Gamer ASUS
Description	Laptop gamer con RTX 3060 y 16GB RAM.
Price	1500
Rating	3
Stock	0
Ensure this value is greater than or equal to 1.	
Brand	ASUS
Thumbnail	https://dlcdnwebimgs.asus.com/gain/3D241166-0518-4745-B481-D901886BFD14
Category	laptops
POST	

7. Ahora probemos a validar desde la shell.

- **python manage.py shell**

8. Procedemos a crear una subasta que incumpla la validación recién impuesta. Ejecuta la siguiente línea:

```
from auctions.models import Category, Auction
from django.utils import timezone
from datetime import timedelta

categ = Category.objects.get(id=2)

auction = Auction(
    title="Laptop Gamer ASUS",
    description="Laptop gamer con RTX 3060 y 16GB RAM.",
    price=1500.00,
    rating=4.8,
    stock=0,
    brand="ASUS",
    category=categ,
    thumbnail="https://dlcdnwebimgs.asus.com/gain/3D241166-0518-4745-B481-D901886BFD14",
    closing_date=timezone.now() + timedelta(days=16)
)

auction.full_clean() #Comando que ejecuta las validaciones
auction.save()
```

9. Nos devolverá el mismo mensaje de error que nos ha devuelto al ejecutarlo mediante api.

```
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "C:\Users\...\djangoServer\Lib\site-
packages\django\db\models\base.py", line 1651, in full_clean
    raise ValidationError(errors)
django.core.exceptions.ValidationError:
{'stock': ['Ensure this value is greater than or equal to 1.']}
```

Ejercicio 1.

Desde la shell de Django, intenta crear de nuevo la subasta y comprueba que pasa si el valor del stock es ahora 10 en lugar de 0. ¿Qué mensaje devuelve el comando ***auction.full_clean()***?

Ejercicio 2.

El campo *rating* corresponde con la valoración que se le da a una subasta. Este valor debe tener un rango comprendido entre 1 y 5. Configura los validadores de modelo necesarios. Puedes encontrar el validador de valor máximo por defecto [aquí](#).

Validaciones en los serializadores

En Django REST Framework (DRF), las validaciones en los serializadores aseguran que los datos sean correctos antes de procesarlos. Son más robustas que las validaciones del modelo porque se aplican incluso antes de intentar guardar los datos en la base de datos.

Es el tipo de validación adecuada si quieres mensajes de error personalizados para el usuario. También cuando la validación depende de múltiples campos.

Probaremos a restringir el campo fecha de cierre de la subasta (*closing_date*). Este deberá ser posterior a la fecha actual.

1. Accede a [auctions/serializers.py](#) y añade las líneas en negrita.

```
class AuctionListCreateSerializer(serializers.ModelSerializer):
    #Resto de campos

    def validate_closing_date(self, value):
        if value <= timezone.now():
            raise serializers.ValidationError("Closing date must be greater
than now.")
        return value

class AuctionDetailSerializer(serializers.ModelSerializer):
    #Resto de campos

    def validate_closing_date(self, value):
        if value <= timezone.now():
            raise serializers.ValidationError("Closing date must be greater
than now.")
        return value
```

- Navega a <http://127.0.0.1:8000/api/auctions/> y crea una subasta con fecha de cierre de 2024. Podéis usar los valores que queráis para el resto de campos.
- El error que os mostrará es el siguiente:

```
Auction List Create

POST /api/auctions/

HTTP 400 Bad Request
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "closing_date": [
    "Closing date must be greater than now."
  ]
}
```

Closing date 24/03/2024 21:49 

Closing date must be greater than now.

Title Laptop Gamer ASUS v3

Description Laptop gamer con RTX 3060 y 16GB RAM.

Price 1500.00

Rating 3.00

Stock 10

Brand ASUS

Thumbnail <https://dlcdnwebimgs.asus.com/gain/3D241166-0518-4745-B481-D901886BFD14>

Category laptops

POST

- Posteriormente, comprobaremos cómo actúa esta validación desde la shell.
 - python manage.py shell**
- Procedemos a crear una subasta que incumpla la validación recién impuesta. Ejecuta la siguiente línea:

```
from auctions.models import Category, Auction
from django.utils import timezone
from datetime import timedelta

categ = Category.objects.get(id=2)

auction = Auction(
    title="Laptop Gamer ASUS",
    description="Laptop gamer con RTX 3060 y 16GB RAM.",
    price=1500.00,
    rating=4.8,
    stock=10, #OJO: stock mayor a 0
    brand="ASUS",
    category=categ,
    thumbnail="https://dlcdnwebimgs.asus.com/gain/3D241166-0518-4745-B481-D901886BFD14",
    closing_date=timezone.now() - timedelta(days=16) #OJO: es resta
)

auction.full_clean() #Comando que ejecuta las validaciones
auction.save()
```

6. Ahora no saltará la validación. Esto se debe a que no estamos a nivel de modelo si no de serializador. Al crear un registro directamente en base de dato son se aplican las validaciones del serializador. Sólo cuando se llaman mediante el servicio api rest.

Ejercicio 3

Modifica la función `validate_closing_date()` para que además compruebe que la diferencia entre la fecha de creación y la fecha de cierre de la subasta es al menos de 15 días.

Debéis probar esta validación creando una nueva subasta con una fecha de cierre inferior a 15 días. Luego, intentad editar la subasta con id 10 para que su fecha de cierre sea inferior a 15 días.

Pista 1: Puedes añadir varios ifs dentro de la función `validate_closing_date()`.

Pista 2: En el serializador, si estamos creando una subasta, la fecha de creación aún no se ha informado. Esto se debe a que es un campo que rellena la base de datos. Podéis usar `"timezone.now()"` como equivalente. Por otro lado, en la edición de una subasta, sí que ya existe el campo.

Validaciones en las vistas

Las validaciones en la vista (views) permiten verificar y restringir datos antes de ser procesados por los serializadores o guardados en la base de datos.

Para probar los validadores de vista modificaremos la vista de listar subastas. Ahora permitirá filtrar por aquellas subastas que contengan en el título o en la descripción un parámetro de entrada (**search**).

1. Accede a [auctions/views.py](#) y añade las líneas en negrita.

```
from django.db.models import Q
#resto de imports

#resto de vistas
class AuctionListCreate(generics.ListCreateAPIView):
    serializer_class = AuctionListCreateSerializer

    def get_queryset(self):
        queryset = Auction.objects.all()
        params = self.request.query_params

        search = params.get('search', None)

        if search:
            queryset =
                queryset.filter(Q(title__icontains=search) |
                               Q(description__icontains=search))
        return queryset
```

Gracias al anterior código, la vista podrá utilizar un queryparam (**search**). En lugar de devolver toda la lista de subastas, filtrará por aquellas que tengan en el título o en la descripción la palabra buscada.

2. Probémoslo:

- **`python manage.py runserver`**
- <http://127.0.0.1:8000/api/auctions/?search=iphone>

3. El parámetro “search” es opcional. Si no se incluye, nos sigue devolviendo la lista completa de subastas.

- <http://127.0.0.1:8000/api/auctions/>

4. A continuación, vamos a añadir una validación para que el parámetro **search**, si viene informado, tenga un tamaño mayor a 3 caracteres.

```
from rest_framework import generics, status
from rest_framework.exceptions import ValidationError
#resto de imports

#resto de vistas
class AuctionListCreate(generics.ListCreateAPIView):
    serializer_class = AuctionListCreateSerializer

    def get_queryset(self):
        queryset = Auction.objects.all()
        params = self.request.query_params

        search = params.get('search', None)
        if search and len(search) < 3:
            raise ValidationError(
                {"search": "Search query must be at least 3 characters long."},
                code=status.HTTP_400_BAD_REQUEST
            )

        if search:
            queryset = queryset.filter(Q(title__icontains=search) |
Q(description__icontains=search))
        return queryset
```

En este código se está comprobando que si el campo **search** viene informado (recordemos que es un parámetro opcional), su longitud sea mayor de 3. Si no lo es, lanza un **ValidationError** con un mensaje personalizado y un código de estado 400.

5. Comprueba su funcionamiento:

- <http://127.0.0.1:8000/api/auctions/?search=ip>

Ejercicio 4

¿Qué esperarías obtener si accedes a esta url: <http://127.0.0.1:8000/api/auctions/?search=>?

Ejercicio 5

Amplia los filtros del listado de subastas para que incluya la búsqueda por categoría y por rango de precios. Estos filtros deben ser acumulativos. Es decir, podrías filtrar por los tres a la vez. Se deben aplicar las siguientes validaciones:

- La categoría por la que se filtra debe ser uno de los valores disponibles en la base de datos.
- El precio mínimo y máximo deben ser números naturales (mayor que 0). El precio máximo debe ser mayor que el precio mínimo.